

Memòria de la Pràctica 2

Aprenentatge per reforç

Serafi Nebot Ginard
Jaume Galmés Ramis

21722 - Intel·ligència Artificial
Grau d'Enginyeria Informàtica
Curs 2025/2026

Índex

1	Introducció	1
2	Disseny algorismes	2
2.1	Algorismes Aprenentatge	2
2.1.1	Monte Carlo	2
2.1.2	SARSA	2
2.1.3	Q-learning	2
2.2	Algorismes Cerca	3
2.2.1	Genètic	3
2.2.2	Programació dinàmica	4
3	Estudi dels paràmetres	5
3.1	Algorismes d'aprenentatge	5
3.2	Algorismes de cerca	9
4	Anàlisi rendiment	13
5	Anàlisi polítiques	16
6	Conclusió	19
7	Intel·ligències Artificials Generatives	20

Introducció

Disseny algorismes

2.1 Algorismes Aprenentatge

Un aspecte particular de la nostra implementació d'aquest tipus d'algorismes és la forma en que es selecciona l'acció a realitzar donat un estat. Donada una taula d'estat-acció $Q(s, a) \in \mathbb{R}$ es genera una política $\pi(s) \in \mathbb{A}(s)$ de la següent forma:

$$\pi(S_t) \leftarrow \arg \max_a Q(S_t, a), \forall S_t \in \mathbb{S}$$

El tema és que nosaltres inicialitzam la taula $Q(\cdot, \cdot) \leftarrow 0$ el que implica que les primeres iteracions totes les accions tenen el mateix valor. Això és un problema perquè la implementació de $\arg \max$ de Numpy sempre retornara el primer element màxim. En el nostre cas les accions estan prioritzades en el següent ordre $\{\text{left, down, right, up}\}$. Per preocupació de que aquesta preferència pugui provocar un biaix en les polítiques resultants hem decidit introduir un *tie-break* aleatori.

$$\pi(S_t) \sim \text{Unif}\left(\arg \max_a Q(S_t, a)\right), \forall S_t \in \mathbb{S}$$

2.1.1 Monte Carlo

2.1.2 SARSA

$$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma Q(S', A') - Q(S, A)]$$

2.1.3 Q-learning

$$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$$

2.2 Algorismes Cerca

2.2.1 Genètic

Individu

Hem decidit que un individu representi una política $\pi_i(S_t) \in \mathbb{A}(S_t)$ enlloc que tota una taula d'estat-acció $Q(S, A) \in \mathbb{R}$ perquè l'espai que ocupa i el temps d'execució son molt menors.

La població d'individus conté, per a cada generació, *population_size* individus. Per a la primera generació, els gens de tots els individus s'inicialitzen de forma completament aleatòria.

Funció *fitness*

La funció *fitness* és la tasa d'èxit de l'individu. Al ser un entorn estocàstic s'evalua la política durant 100 episodis i es calcula el promig de vegades que s'ha arribat al destí.

Pel cas particular del *Frozen Lake* totes les recompenses són 0 excepte quan s'arriba al destí, per tant, la funció *fitness* és equivalent al promig de recompenses totals obtingudes al llarg de tots els episodis (*Success Rate*).

$$fitness(\pi_i) = \frac{1}{100} \sum_{j=1}^{100} \sum_{t=1}^T R_t^j$$

On R_t^j és la recompensa del període t de l'episodi j .

Elitisme o *Culling*

Abans de generar la nova generació es seleccionen els millors individus que passen directament a la nova generació (la resta es descarten). La quantitat d'individus es calcula de la següent forma:

$$best_size = \lceil culling_rate \cdot population_size \rceil$$

On $culling_rate \in [0, 1]$ és la tasa de *culling*. Un $culling_rate > 0$ ens garanteix que no es perdin els millors individus amb el pas de les generacions.

Selecció

De tots els individus de cada generació (*population_size*) només es seleccionen els *parents_size* millors:

$$parents_size = \lceil selection_pressure \cdot population_size \rceil$$

On $selection_pressure \in (0, 1]$ és el paràmetre de pressió de selecció. D'aquesta forma, $selection_pressure$, ens permet escollir quin percentatge de la millor part de la població volem emprar per a generar la pròxima generació.

Mutació

Després de generar cada individu hi ha una petita probabilitat $mutation_rate$ de mutar cada gen de forma aleatòria.

$$nou_gen = \begin{cases} \text{original} & \text{si } p \geq mutation_rate \\ \text{aleatori} & \text{si } p < mutation_rate \end{cases}$$

On $p \in [0, 1]$ és un nombre aleatori generat per a cada gen.

2.2.2 Programació dinàmica

Estudi dels paràmetres

A continuació es mostra com diferents variacions dels paràmetres de cada un dels models afecta als resultats.

Per a cada configuració dels paràmetres, cada un dels algorismes primer passa per a la fase d'entrenament de n iteracions i després s'evalua la política resultant amb 1000 episodis per a contrarestar el factor estocàstic de l'entorn.

Per a mantenir el document breu, les úniques mètriques que comparam són el nombre promig de passes (*steps*) que es tarda per a cada episodi i el promig de vegades que s'ha arribat al destí, anomenat *sr* (*Success Rate*).

3.1 Algorismes d'aprenentatge

Per a tota aquesta secció el nombre d'episodis d'entrenament $n = 20000$.

Per a *Monte Carlo*, *SARSA* i *Q-learning* tenim els següent paràmetres:

- $dr = 0.99$: *Discount Rate* o *gamma* als apunts, és el factor de descompte per a la recompensa esperada per a cada passa del mateix episodi.
- $er = 1.00$: *Exploration Rate* o *epsilon* als apunts, és el factor d'exploració per a una política *epsilon-greedy*.
- $er_min = 0.01$: *Exploration Rate Minimum* és el mínim permès del valor er .
- $er_decay = 0.9995$: *Exploration Rate Decay* és el factor de descens del paràmetre er . A cada iteració del bucle d'entrenament es decrementa el factor d'exploració: $er = \min\{er_min, er \cdot er_decay\}$.

Per a *SARSA* i *Q-learning* tenim $lr = 0.01$ (*Learning Rate* o *alpha* als apunts) que és el factor d'aprenentatge.

Amb aquests valors per defecte dels paràmetres obtenim els següents resultats que ens serviran per a comprar amb el pròxim apartat:

Monte Carlo		SARSA		Q-learning	
steps	sr	steps	sr	steps	sr
41.3870	0.7030	44.3640	0.7320	44.1830	0.7300

A continuació es mostren els resultats de totes les variacions de cada un dels paràmetres:

	Monte Carlo		SARSA		Q-learning	
dr	steps	sr	steps	sr	steps	sr
0.10	15.3360	0.2110	9.2340	0.0620	9.2420	0.1120
0.20	9.2610	0.1030	8.5430	0.1010	7.3150	0.0470
0.30	6.9370	0.1010	16.0000	0.1760	6.2980	0.0600
0.40	8.4560	0.1020	20.9020	0.1790	14.2070	0.1560
0.50	10.0620	0.1180	14.5330	0.1570	21.9130	0.2800
0.60	6.1020	0.0750	33.7380	0.5140	22.0330	0.2630
0.70	35.7620	0.5640	16.2900	0.2180	30.5880	0.4590
0.80	18.2380	0.2030	26.8600	0.3900	27.9760	0.4460
0.90	8.5550	0.0670	41.5030	0.6280	41.8290	0.7180
1.00	26.0580	0.3240	44.7220	0.7600	100.000	0.0000

SARSA i Q-learning es comporten de forma molt similar i prefereixen un dr molt proper a 1, exceptuant el cas $dr = 1.00$ on Q-learning es veu molt afectat i no arriba cap vegada al destí en totes les 1000 iteracions de prova.

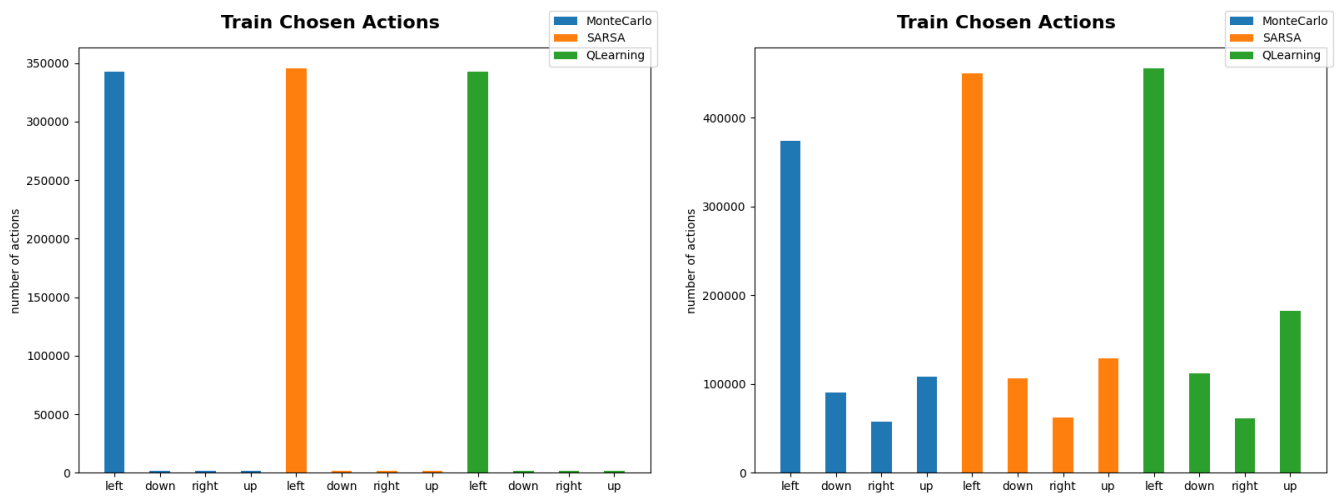
Monte Carlo, per altra banda, sembla ser molt sensible a petits canvis del paràmetre i, per aquesta prova limitada, no podem veure un patró clar. A les proves que hem fet durant el desenvolupament sabem que el valor per defecte que hem triat genera resultats variats (0.30 i 0.70 de *Survival Rate*, que és el que veiem a la taula amb el valor 1.00), dependent de la inicilització de la semilla del generador de nombres aleatoris.

	Monte Carlo		SARSA		Q-learning	
er	steps	sr	steps	sr	steps	sr
0.10	24.7740	0.2420	43.8000	0.7430	46.1800	0.7360
0.20	27.6060	0.1950	45.8280	0.7440	43.3920	0.7630
0.30	38.0640	0.6740	43.5510	0.7550	45.3150	0.7380
0.40	30.1160	0.5200	45.4450	0.7440	45.6250	0.7290
0.50	40.1320	0.7390	43.4110	0.7570	45.3650	0.7710
0.60	30.0940	0.2350	43.7320	0.7640	45.3130	0.6970
0.70	40.6120	0.7150	43.8760	0.7380	44.4060	0.7330
0.80	40.6070	0.7340	44.9870	0.7380	44.1360	0.7530
0.90	41.1670	0.7310	42.6170	0.7520	45.2820	0.7320
1.00	41.2560	0.7030	45.6850	0.7440	45.0200	0.7330

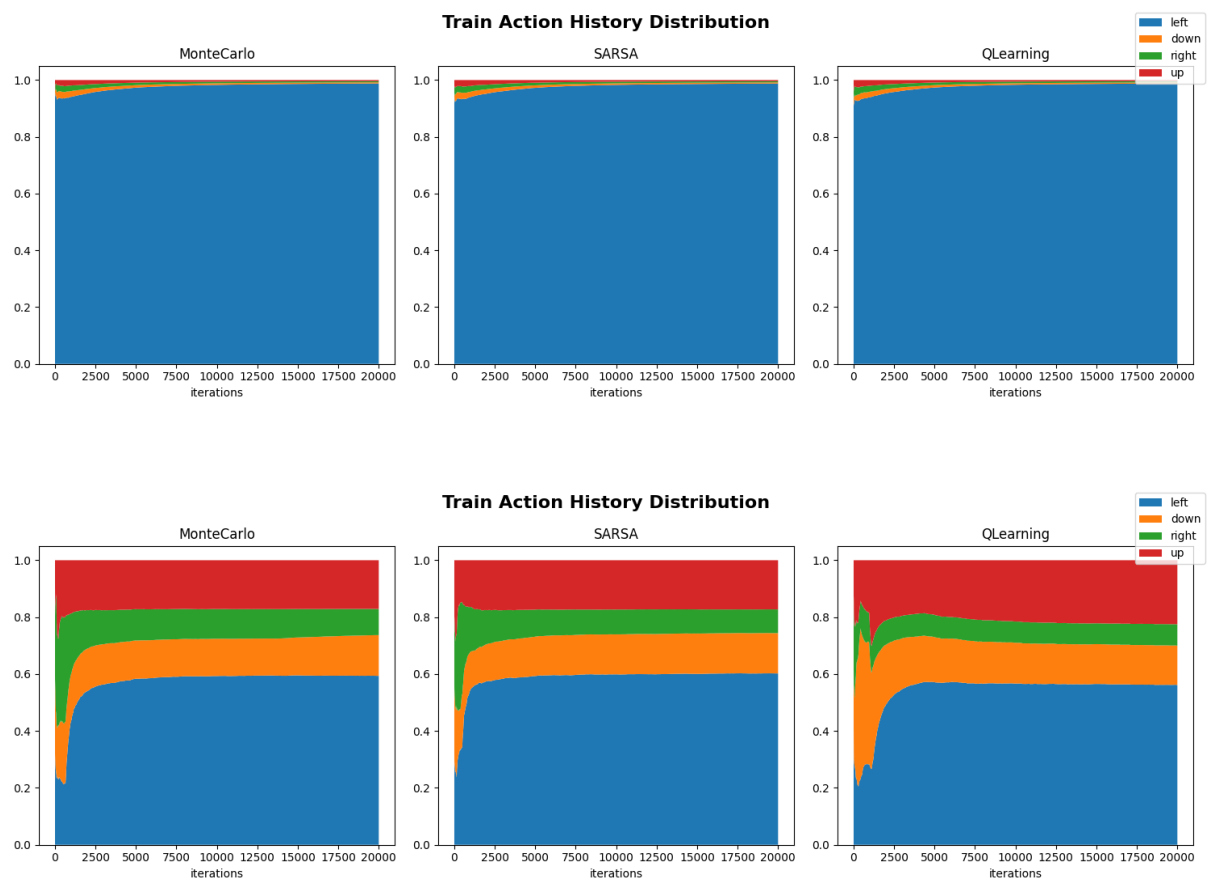
Monte Carlo necessita una exploració major a l'inici de l'entrenament per a poder trobar una solució òptima.

SARSA i Q-learning no es veuen afectats pel valor de er . Això és degut a que la nostra implementació d'aquests algorismes utilitza un *tie-break* aleatori. Si tornam a entrenar els

algorismes amb $er = 0.10$ però activant/desactivant el *tie-break* i graficam la distribució de les vegades que s'ha escollit cada acció durant la fase d'entrenament tenim (esquerra *tie-break off*, dreta *tie-break on*):



També podem graficar l'evolució de la distribució al llarg de l'entrenament (el primer amb *tie-break off* i el segon amb *tie-break on*):



A simple vista es veu que hi ha una diferència abismal. Quan el *tie-break* està desactivat i l'exploració és molt baixa l'acció *left* està desproporcionadament afavorida. Quan el *tie-break* està activat l'elecció de les accions està molt més distribuïda. La raó perque

encara hi ha una preferència sobre les accions *left* i *up* es discuteix a l'apartat d'Anàlisi de les polítiques.

	Monte Carlo		SARSA		Q-learning	
<i>er_min</i>	steps	sr	steps	sr	steps	sr
0.10	6.3470	0.0660	40.7290	0.7200	45.1450	0.7380
0.20	10.7840	0.0900	39.1340	0.7460	45.5910	0.7680
0.30	27.8450	0.2690	41.3910	0.7160	44.2060	0.7570
0.40	6.7520	0.0980	41.9450	0.7300	43.6270	0.7450
0.50	12.3250	0.1870	39.7820	0.6890	43.4790	0.7480
0.60	11.2170	0.1170	42.2630	0.7120	45.9040	0.7170
0.70	9.0570	0.1260	23.1390	0.2830	44.0580	0.7150
0.80	32.8600	0.5210	39.8010	0.7070	44.8900	0.7240
0.90	24.2700	0.3700	36.0990	0.6080	44.2430	0.7470
1.00	34.0410	0.4590	24.2430	0.3300	45.1970	0.7510

Q-learning no es veu afectat en absolut i SARSA negativament per a valors alts de *er_min*. Per tant, una exploració alta al final de la fase d'entrenament perjudica molt a SARSA probablement degut a que per a calcular la correcció de la recompensa esperada SARSA utilitza l'acció realitzada i Q-learning sempre utilitza la millor, ignorant així el factor aleatori.

Monte Carlo, per altra banda, prefereix una exploració alta mantinguda, però segons els paràmetres per defecte que hem triat sabem que millora encara més.

	Monte Carlo		SARSA		Q-learning	
<i>er_decay</i>	steps	sr	steps	sr	steps	sr
0.10	8.0950	0.0810	45.0600	0.7450	44.2680	0.7310
0.20	24.4520	0.2640	44.3870	0.7490	44.5310	0.7350
0.30	27.6170	0.4670	44.1920	0.7530	45.1240	0.7410
0.40	30.0460	0.2070	44.2520	0.7570	44.5730	0.7040
0.50	10.5710	0.0960	43.6780	0.7320	43.7620	0.7330
0.60	24.0870	0.1480	43.4670	0.7380	43.2330	0.7310
0.70	13.6050	0.1610	45.8600	0.7550	45.3060	0.7320
0.80	38.2980	0.6750	46.0460	0.7290	42.9090	0.7590
0.90	9.6760	0.0970	44.9890	0.7380	44.9960	0.7310
1.00	39.8340	0.6960	22.6820	0.3380	44.5930	0.7470

Una altre vegada, Q-learning no es veu afectat en absolut i SARSA només quan es manté el factor d'exploració alt (el mateix per a *er_decay* = 1.0) durant tot l'entrenament.

En aquest cas, Monte Carlo obté molt bons resultats quan *er_decay* és molt proper a 1.0, consistent amb els resultats que hem obtingut amb el valor per defecte.

	SARSA		Q-learning	
<i>lr</i>	steps	sr	steps	sr
0.10	44.607	0.74	44.711	0.721
0.20	44.706	0.644	45.731	0.745
0.30	44.441	0.72	45.358	0.732
0.40	46.921	0.638	45.021	0.725
0.50	46.462	0.734	43.33	0.734
0.60	28.884	0.244	44.917	0.735
0.70	62.305	0.561	42.406	0.756
0.80	61.438	0.573	44.273	0.758
0.90	8.856	0.102	45.38	0.744
1.00	7.832	0.024	7.837	0.022

Podem veure que SARSA és molt més sensible al *Learning Rate* que Q-learning, ja que aquest darrer es manté igual al llarg de totes les combinacions.

Per a valors molt grans de *lr* ambdós tenen resultats molt pobres, sobretot quan son molt propers a 1. Això té sentit perquè significa que per a cada actualització de la política s'està corregint un 100% de l'error, es a dir, s'estàn fent salts molt grans i no s'arriba a la solució òptima (s'arriba a un màxim local enlloc d'un màxim global).

3.2 Algorismes de cerca

Per a l'agorisme Genètic tenim els següent paràmetres:

- *population_size* = 100: Nombre total d'individus a la població per a cada generació.
- *selection_pressure* = 0.50: Percentatge de la població que es selecciona per a generar la pròxima generació, la resta es descarten.
- *mutation_rate* = 0.08: Probabilitat de mutar cada un dels gens de cada individu nou generat.
- *culling_rate* = 0.01: Percentatge de la població que passa directament a la següent generació (també conegut com a elitisme).

Amb aquests valors per defecte dels paràmetres obtenim els següents resultats que ens serviran per a comprar amb el pròxim apartat:

Genètic	
steps	sr
40.5680	0.6920

	Genètic	
<i>population_size</i>	steps	sr
1	5.4120	0.0240
10	27.7020	0.1930
100	43.0160	0.7150
1000	40.1770	0.7340
10000	41.9850	0.7250

Hi ha una tendència molt clara amb la quantitat d'individus dintre de la població: quants més n'hi ha millor és el resultat.

Els resultats es saturen i ja no milloren quan s'atraquen als 100 individus. Al ser un entorn molt petit i limitat, tenir una població molt gran no proporciona molta millora. Potser que per a entorns on els individus tinguin més gens una població més gran permeti una població amb més varietat i en conseqüència generi millors resultats.

	Genètic	
<i>selection_pressure</i>	steps	sr
0.10	44.8740	0.7330
0.20	45.5660	0.7360
0.30	41.9680	0.7250
0.40	41.2320	0.7200
0.50	41.9620	0.7250
0.60	39.4190	0.7040
0.70	41.6150	0.7000
0.80	39.3180	0.7020
0.90	38.7690	0.5880
1.00	41.8970	0.7410

Resultats bàsicament idèntics. Només hi ha un *Survival Rate* per davall dels altres que probablement sigui degut a la inicialització de la semilla del generador de nombres aleatoris.

Al igual que la mida de la població, segurament amb un problema més complex hi hauria una variació major.

	Genètic	
<i>mutation_rate</i>	steps	sr
0.00	42.5690	0.7660
0.10	41.9430	0.7370
0.20	41.2760	0.7230
0.30	41.9580	0.7210
0.40	37.6350	0.6730
0.50	41.7470	0.5280
0.60	34.6640	0.4390
0.70	41.5350	0.5550
0.80	29.5650	0.3280
0.90	32.7790	0.4000
1.00	23.2970	0.2750

Resultats molt millors per a un *mutation_rate* menor. Té sentit perquè com major sigui aquest valor més s'atraca a ser una cerca aleatoria.

	Genètic	
<i>culling_rate</i>	steps	sr
0.00	41.7680	0.7070
0.10	38.1680	0.7420
0.20	44.6360	0.7440
0.30	41.5580	0.7150
0.40	42.8560	0.7330
0.50	41.6510	0.7350
0.60	39.6660	0.7160
0.70	41.0450	0.7300
0.80	40.8990	0.7040
0.90	45.2610	0.7380
1.00	41.0240	0.6990

Resultats bàsicament constants. Al igual que la mida de la població i la pressió de selecció, aquest és un entorn massa senzill i amb algú de més complexe potser es vegessin els canvis.

Per a l'agorisme de Programació Dinàmica només tenim el paràmetre $dr = 0.95$ (*Discount Rate* o *gamma* als apuunts) és el factor de descompte per a la recompensa esperada per a cada passa del mateix episodi.

Amb aquests valors per defecte dels paràmetres obtenim els següents resultats que ens serviran per a comprar amb el pròxim apartat:

Prog. Dinàmica	
steps	sr
39.7750	0.7340

	Prog. Dinámica	
dr	steps	sr
0.10	27.4400	0.4560
0.20	27.7030	0.4160
0.30	27.0910	0.4560
0.40	28.0500	0.4560
0.50	28.3110	0.4450
0.60	27.0930	0.4220
0.70	27.2700	0.4430
0.80	27.7850	0.4250
0.90	39.7830	0.7520
1.00	40.0280	0.7500

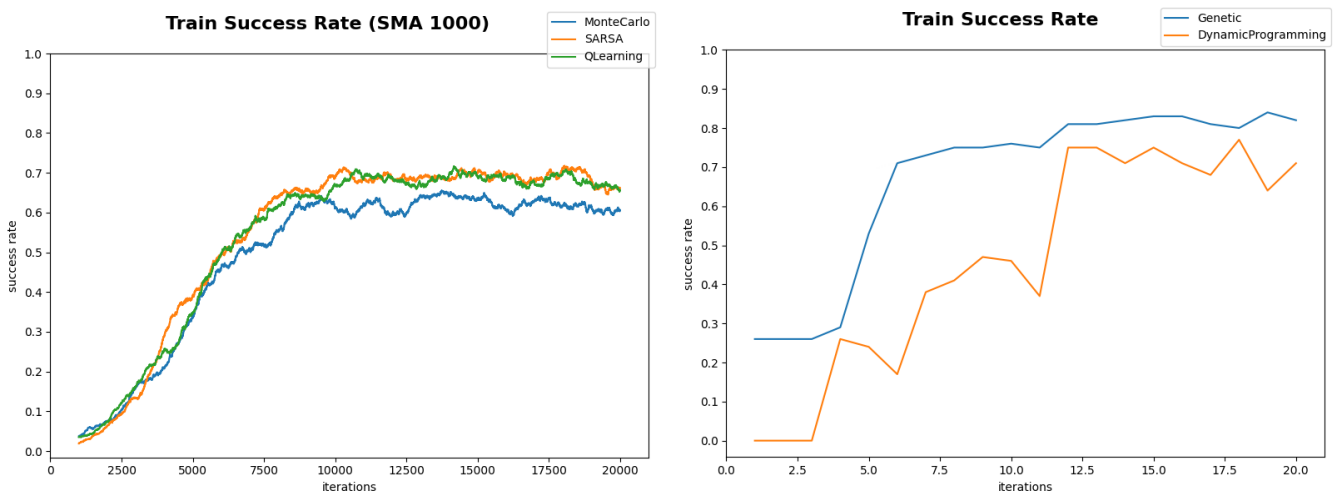
Els resultats es mantenen constants del 0.10 fins al 0.80 i després milloren bastant i es saturen a partir del 0.90. Un *Discount Rate* major té sentit perquè al estar explorant tots els estats possibles ens interessa mantenir tota la informació (sabem que el resultat és correcte perquè els hem explorat tots).

Anàlisi rendiment

Com que els algorismes d'aprenentatge necessiten una quantitat molt elevada d'episodis per a la fase d'entrenament i els de cerca en necessiten molt menys (en l'ordre de 1000x menys) es separen aquestes dues categories amb dues gràfiques diferents.

A més, els algorismes d'aprenentatge al necessitar moltes iteracions cal realitzar qualche tipus d'agregació dels valors per a poder obtenir una senyal interpretable. Hem decidit utilitzar una mitjana mòvil dels darrers 1000 valors. En els casos on s'utilitzi s'indica al títol del gràfic amb l'acrònim SMA (*Simple Moving Average*) seguit del valor de la finestra.

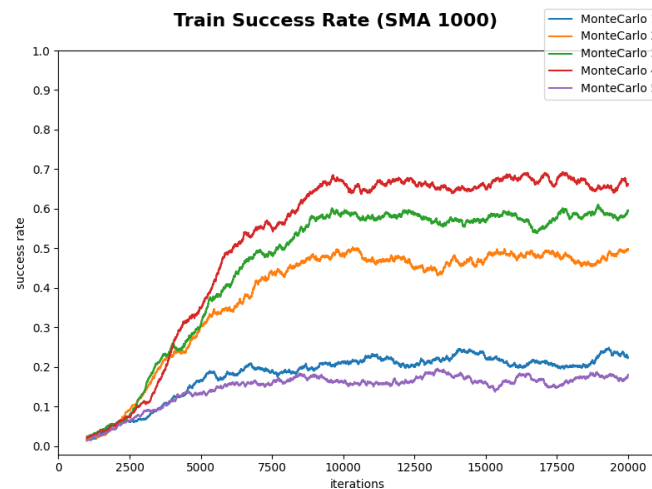
Primer de tot, anem a analitzar l'evolució del rendiment de la política durant el procés d'entrenament. Per fer-ho, utilitzam el *Success Rate* que és el nombre promig de vegades que s'arriba al destí al llarg d'uns quants episodis. Pels algorismes d'aprenentatge s'utilitza el promig de vegades que s'ha arribat al destí dintre la finestra i pels algorismes de cerca s'evalua la política 100 vegades després de cada iteració d'entrenament (ambdós utilitzen una política *epsilon-greedy*; excepte per l'algorisme genètic).



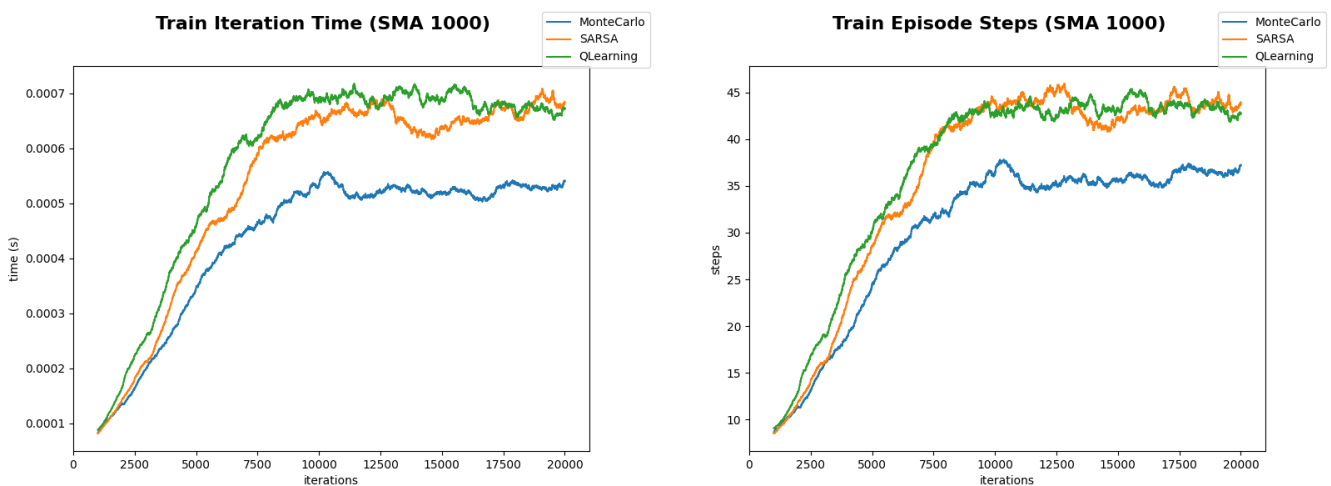
Podem veure com tots els algorismes comencen amb un *Success Rate* molt baix degut a que la seva política és completament aleatòria i no aconsegueixen arribar al destí, milloren molt ràpidament i arriben a un punt de saturació a partir d'un cert nombre d'iteracions a partir del qual ja no milloren més. En el cas particular de la programació dinàmica inclús es podria especular que la millor és lineal, però tampoc ho podem afirmar perquè la quantitat de punts de dades és molt baix.

A partir de les proves que hem anat realitzant durant el desenvolupament sabem que el punt de saturació de l'algorisme Monte Carlo oscila entre els 0.10 i 0.70 de *Success*

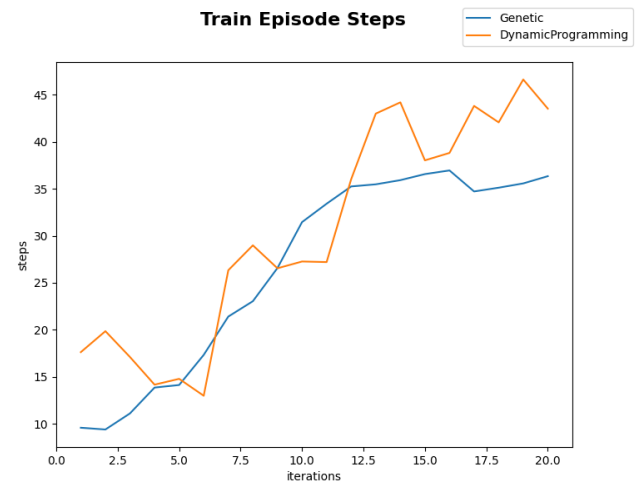
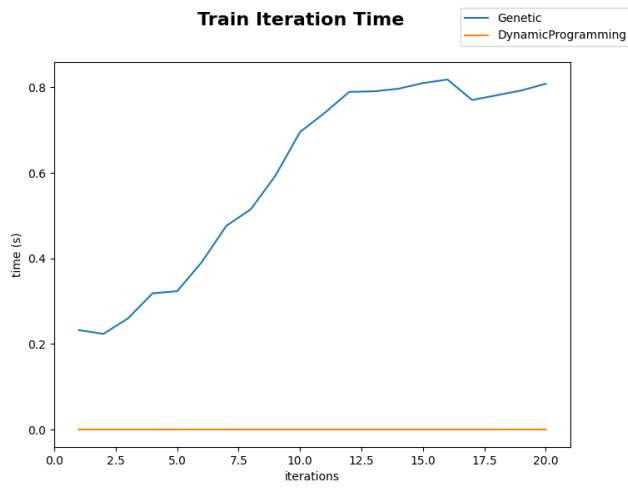
Rate. Això depèn de la inicialització de la semilla del generador de nombres aleatoris i no sabem molt bé com, o si es pot, evitar. Si realitzam l'entrenament de l'algorisme 5 vegades obtenim el següent resultat.



Una altra mètrica a mirar és el temps de cada iteració del bucle d'entrenament, que està directament relacionat amb el nombre de passes que es realitzen a cada episodi. Com que al principi de l'entrenament la política és aleatòria els episodis acaben molt aviat perquè quan es cau a un forat aquest acaba. Així com l'algorisme millora la política i s'evita caure als forats es permeten fer més passes i allargar l'episodi. Per tant, aquestes dues mètriques també estan vinculades al *Success Rate* que hem vist abans.

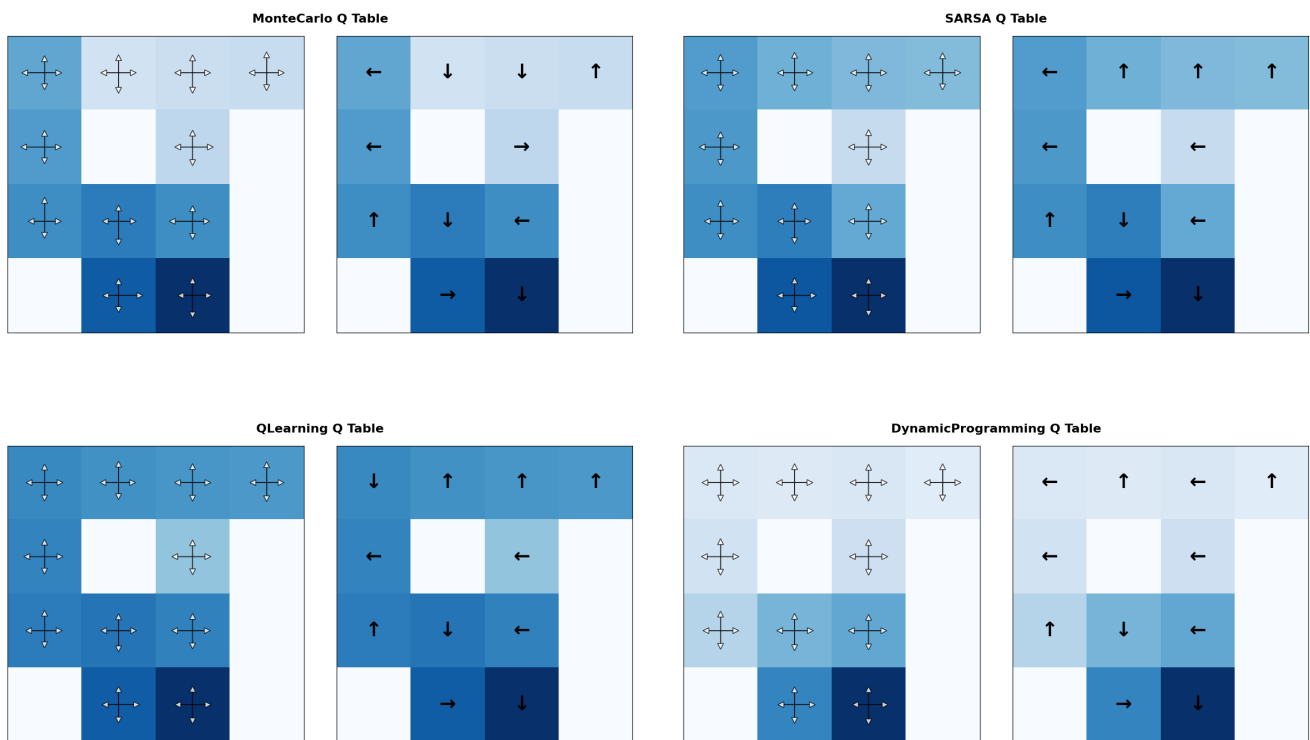


Pel cas de la Programació Dinàmica el temps d'execució de cada iteració d'entrenament és despreciable i es manté constant a uns $65\mu s$. Això és degut a que aquest algorisme no crida a l'entorn de *Frozen Lake* de Gymnasium ja que només necessita la matriu de probabilitats de transició per a cada estat.



Anàlisi polítiques

A continuació es mostren les polítiques per a cada un dels algorismes representada amb dos gràfics. El gràfic de l'esquerra mostra la taula Q. El color de cada cel·la representa el valor de la recompensa esperada per a aquell estat (blau més fort implica una recompensa major) i les fletxes representen les accions amb la seva llargària proporcional al seu valor (una fletxa més llarga té una recompensa esperada més alta que d'una de petita). El gràfic de la dreta mostra el mateix però només amb la fletxa de més valor per a poder distingir quina és la millor opció.



A primera vista, el que ressaltava més és la distribució dels valors sobre el mapa.

- Monte Carlo sembla prioritzar un camí molt més que l'altre (primer moviment per avall enlloc de la dreta).
- SARSA té la mateixa preferència que Monte Carlo però menys exagerada.
- Q-learning té les recompenses molt repartides, producte de la seva naturalesa optimitista.
- Programació Dinàmica centra les recompenses al voltant del destí.

El que pot semblar poc intuïtiu és la millor acció que trien els algorismes. Per exemple, no n'hi ha cap que per a l'estat anterior del final (el que està marcat amb més valor; abaix a la dreta) indiqui l'acció que duu directament cap al destí com a la millor.

Això és degut a que hi ha major probabilitat de lliscar que fer l'acció escollida:

$$p(\text{dreta}) = p(\text{amunt}) = p(\text{avall}) = \frac{1}{3} \implies p(\text{amunt} \cup \text{avall}) = \frac{2}{3} > \frac{1}{3} = p(\text{dreta})$$

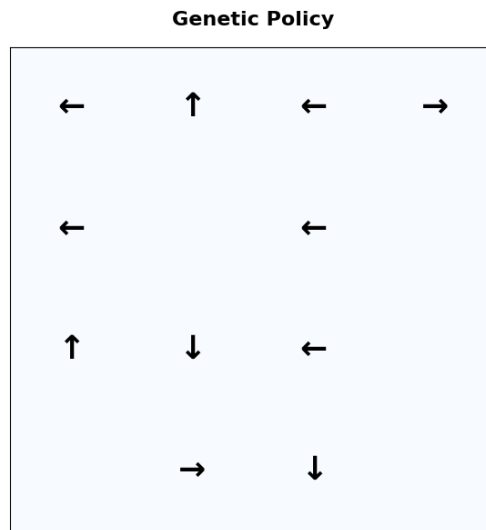
Exactament el mateix passa amb els estats adjacents a forats, ja que alguns models trien l'opció que duu directament al forat com a la millor.

Per a profunditzar un poc més sobre aquest fenomen anem a analitzar la quantitat de vegades que l'algorisme tria una determinada acció durant la fase de l'entrenament (només es mostren els algorismes d'aprenentatge perquè dels altres és impossible saber aquesta dada).

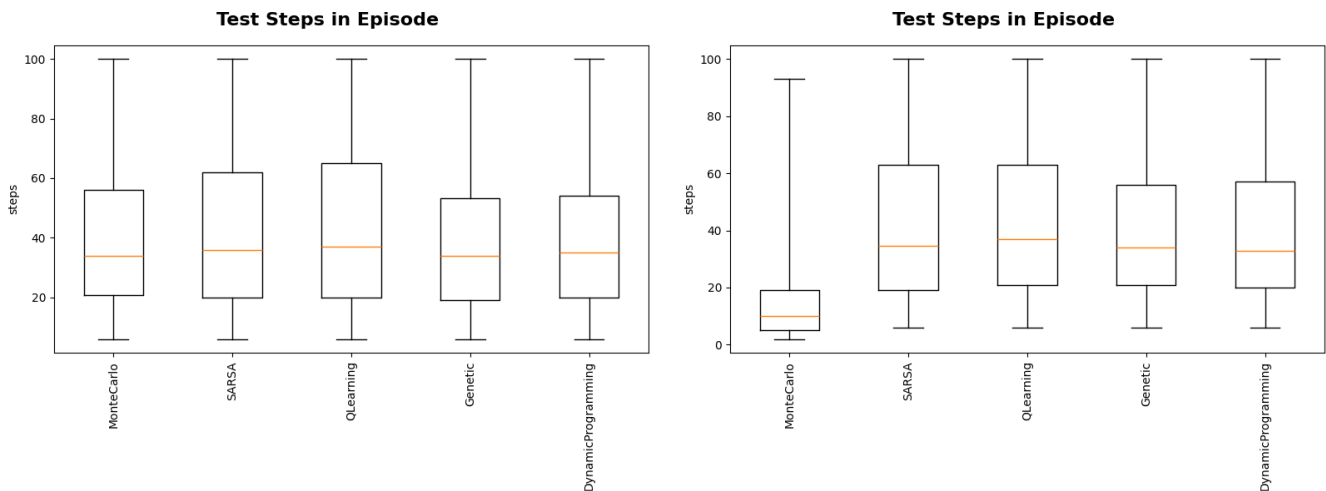


Es veu clarament que per a tots els algorismes hi ha una preferència sobre l'acció *left* seguit de l'acció *up*. Si ho raonem, té sentit que sigui així perquè l'episodi sempre comença a dalt de tot a l'esquerra i, com que sortir-se del tauler no provoca el fi de l'episodi, és preferible anar cap a fora (*left* o *up*) i així evitar caure a un forat en el cas de que es llisqui.

El darrer algorisme, el Genètic, al calcular directament la política i no calcular els valors d'estat-acció només es poden mostrar les fletxes (accions) i, per tant, no es pot fer un anàlisi tant detallat. Però és interessant veure com, sense calcular els valors d'estat-acció i només utilitzant la "força bruta", s'arriba a una política pràcticament idèntica a molts dels altres algorismes.



També pot ser interessant analitzar la quantitat de passes realitzades per a cada algorisme al llarg de 100 episodis. Es mostren dues versions per tal de mostrar com afecta la qualitat de la política amb el nombre de passes per episodi. El gràfic de l'esquerra pertany a una execució on totes les polítiques aconseguixen un *Success Rate* del 0.70 i a la dreta on Monte Carlo només ha aconseguit un 0.10 i la resta es mantenen igual.



Es veu clarament que una millor política implica més passes (per les mateixes raons comentades a l'apartat d'Anàlisi del rendiment). Curiosament, SARSA i Q-learning semblen generar unes polítiques lleugerament més conservadores (més passes significa que s'ha anat més alerta a caure).

Conclusió

L'entorn *Frozen Lake* de Gymnasium tot i ser un entorn molt simple hem vist com el problema es complica molt quan s'activa l'opció *slippery* (passam d'un entorn completament determinista a un que no ho és). Hem posat a prova tots els algorismes vists a teoria, hem pogut veure les forces i febleses de cada un d'ells i l'importància de cada un dels seus paràmetres.

En particular hem vist que Monte Carlo és molt inconsistent amb els resultats que proporciona. Potser això es pugui solucionar amb diferents combinacions dels paràmetres però no hem tengut el temps per a investigar-ho.

Hem pogut profunditzar bastant en el funcionament dels algorismes i hem vist com la decisió originalment “preventiva” del *tie-break* aleatori a l'anàlisi dels paràmetres, cosa que no ens esperavem.

Interessant, també, veure com les polítiques resultants poden ser contraintuïtives (la millor opció anar directament cap a un forat) i com hi ha accions molt més preferibles que altres.

Com a opinió personal, podem dir que és un projecte que ha posat a prova el que sabem i ens ha forçat a repassar i expandir conceptes que hem vists a teoria, cosa molt important a l'hora d'assolir coneixements. A la part d'anàlisi haguéssim pogut profunditzar o complicar més però amb el que hem fet ja hem anat molt justs de temps.

Intel·ligències Artificials Generatives

Al llarg del desenvolupament del projecte hem emprat les següents intel·ligències artificials generatives:

- ChatGPT-5.1
 - Validació i correcció algorismes implementats amb Python
 - Recomenació configuració paràmetres per a diferents algorismes
 - Correcció notació matemàtica utilitzada a la memòria
- Grok 4.1:
 - Validació i correcció algorismes implementats amb Python
 - Recomenació configuració paràmetres per a diferents algorismes
- Perplexity (ChatGPT-5.1, Claude Sonnet 4.5 o Sonar):
 - Cerca documentació i exemples Python, Numpy i Matplotlib

Pràctica II

Intel·ligència Artificial

Aprenentatge per reforç

Serafi Nebot Ginard
Jaume Galmés Ramis

Grau d'Enginyeria Informàtica
Curs 2025/2026